

Introduction to Machine Learning and Deep Learning with Python

Day 2

CNRS Formation Entreprise

10 – 12 June, 2026, Paris

Tabea Rebafka, Vincent Lemaire, Daphné Giorgi



Bagging

Random forests

Boosting

Bagging

Random forests

Boosting

General context

- For each individual $i = 1, \dots, n$, we observe
 - a **feature vector** $x_i \in \mathbb{R}^d$ with $x_i = (x_i^1, \dots, x_i^d) = (x_i^j)_{1 \leq j \leq d}$
 - an **outcome** y_i with values
 - in a finite number of classes $\{1, \dots, K\}$ (**classification problem**)
 - in $\mathbb{R}$ (**regression problem**)
- Data $\mathcal{D} = \{(x_i, y_i), i = 1, \dots, n\}$ are considered to be independent realizations of some random (X, Y)
- Goal: given a new feature vector x_{new} , we want to **predict** its unobserved outcome y_{new} , which is either a real value or a class label.
- Approach: **learn the relation** between Y and X to make predictions

Regression models

Model the relationship between Y and X as

$$y_i = f(x_i) + E_i, \quad i = 1, \dots, n$$

where

- f is a **regression function** representing the **systematic part** of the relation
- E_i is an **error term** representing the **random part** of observation y_i
 - E_i is supposed to be small (mean zero, independence)

Linear model

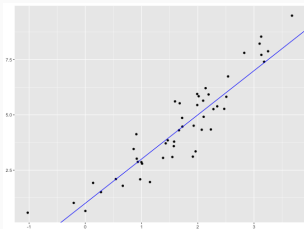
- Function f is assumed to be **linear** of the form

$$f(x) = w^T x + b$$

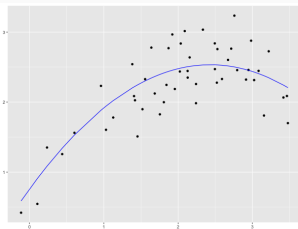
with parameters $w \in \mathbb{R}$ and $b \in \mathbb{R}$ to be estimated

- $\hookrightarrow$ appropriate when the relationship between X and Y is approximately linear (that is, when variables are strongly correlated)
- Variants of the linear model
 - segmented linear regression (= piecewise linear regression)
 - polynomial regression (or other basis functions as wavelets)
 - and others

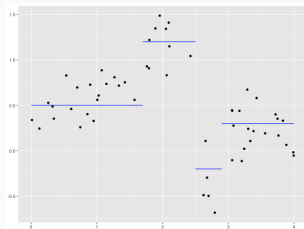
Supervised learning iv



linear



polynomial



piecewise constant

Nonparametric regression models

- A **nonparametric approach** does not make assumptions on the specific form of the regression function f , unlike the linear model
- The family of possible functions f is much richer than in so-called parametric models like the linear model

Many available nonparametric models

- kernel regression
- local regression (LOESS)
- smoothing splines
- k -nearest neighbor algorithm
- **regression trees**
- neural networks

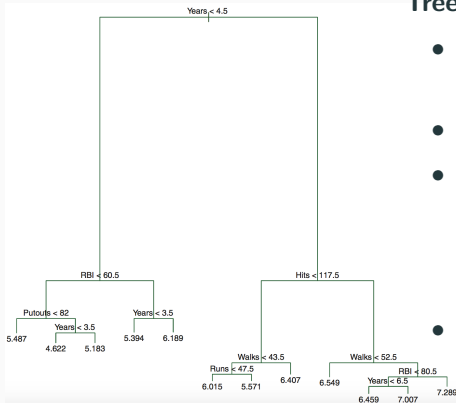
Bagging

Random forests

Boosting

Decision trees for supervised learning i

Regression tree

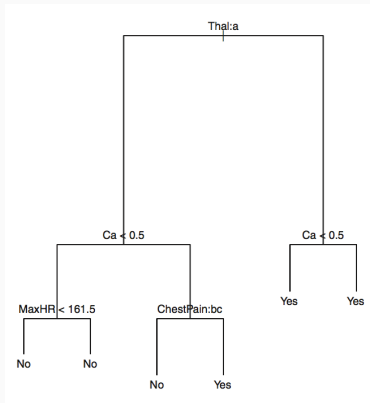


Tree principle for prediction

- Use a tree to make a prediction for x_{new}
- Start at the **root** of the tree
- At every **branching** or **node**, a simple “question” on a single covariable guides the way through the tree
- The value in the **leaf** to which x_{new} arrives is used as the predicted value $\hat{y}_{\text{new}}$

Decision trees for supervised learning ii

Classification tree



Tree principle

- The only difference with regression trees is that leafs contain **class labels**
- Splitting is based on “if-then-else” rules:
 $x^j \leq t$ or $x^j > t$? (for some feature j and threshold t)
- **Binary tree**: exactly two branches at every node

Decision trees for supervised learning iii

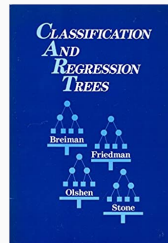
Unified framework

for both types of decision trees:

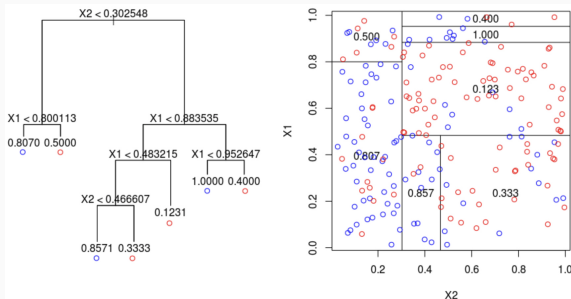
CART = Classification and Regression Trees

Decision trees

- are a **simple** method and
- highly **interpretable**



Construction of decision trees i



values in the leaves/rectangles = proportion of blue training observations in the leaf/rectangle

Decision trees

- represent a **stratification** or **segmentation** of the feature space into a number of simple regions
- the **splitting rules** used to segment the feature space are summarized in the **decision tree**

Construction of a tree

- **Recursive algorithm to partition** the feature space
- **Top-down** construction where branches are created (**branching**)
- Algorithm
 - Start from a single region containing all training data (called **root** of the tree)
 - Recursively choose a feature j and a threshold t to split nodes into ever smaller rectangular regions
- **Splits** are very simple as they only involve a single feature j and a threshold
- Leads to a **guillotine partition** of the feature space
- Which is equivalent to a **binary tree**

Predictive values

- Predictive values in the **leaves** are based on the part of the training data that fall in this leaf
 - regression: **mean value** of training data in the leaf
 - classification context: **majority vote** among labels of training data in the leaf

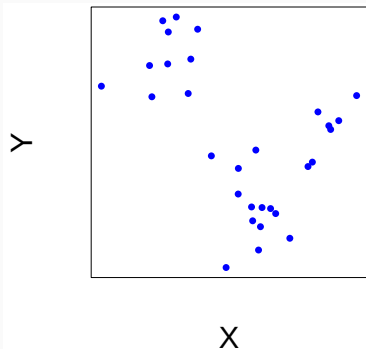
Problem

- How to **construct a decision tree** with high **prediction accuracy**?
- Prediction quality depends on the tree (the partition) and there are many, many possible trees
- Finding the optimal tree is hard!
- How to **make a good split**?

How to split?

- Split such that the labels or outcomes of the training data in the two new regions are as **homogeneous** as possible
 - For classification: the majority of observations in a node should have the same class label
 - For regression: the outcomes in a node should be very concentrated

Quantification of homogeneity

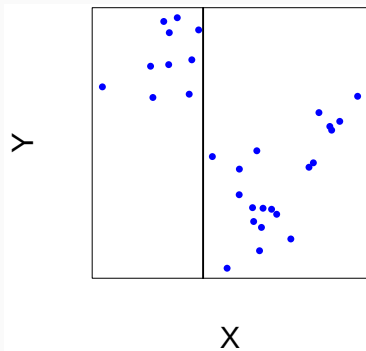


Regression context

- **Variance** of training observations in node $N \subset \{1, \dots, n\}$: with $\bar{y}_N = \frac{1}{\#N} \sum_{i \in N} y_i$ and $\#N$ the number of data points in node N

Make a split iv

Quantification of homogeneity



Regression context

- **Variance** of training observations in node $N \subset \{1, \dots, n\}$:

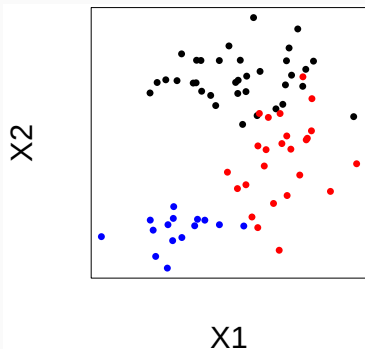
$$V(N) = \frac{1}{\#N} \sum_{i \in N} (y_i - \bar{y}_N)^2$$

with $\bar{y}_N = \frac{1}{\#N} \sum_{i \in N} y_i$ and

$\#N$ the number of data points in node N

Make a split v

Quantification of homogeneity

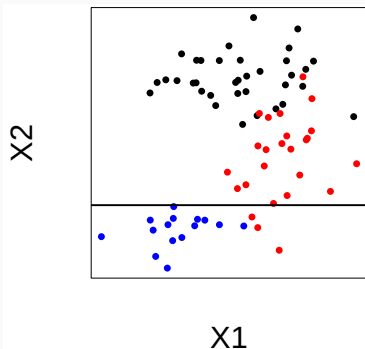


Classification context

- different measures of **node (im)purity**
- all based on **class frequencies** of training observations in node N : For every class k

$$p_{N,k} = \frac{\#\{i \in N : y_i = k\}}{\#N}$$
$$= \frac{\text{number of data points in } N \text{ in class } k}{\text{number of data points in } N}$$

Quantification of homogeneity



Classification context

- different measures of **node (im)purity**
- all based on **class frequencies** of training observations in node N : For every class k

$$p_{N,k} = \frac{\#\{i \in N : y_i = k\}}{\#N}$$
$$= \frac{\text{number of data points in } N \text{ in class } k}{\text{number of data points in } N}$$

Impurity measures for classification

- **Classification error rate**: fraction of the observations in node N that do not belong to the most common class:

$$1 - \max_k p_{N,k}$$

↪ not sufficiently sensitive for growing a tree

- **Gini index**

$$G(N) = \sum_{k=1}^K p_{N,k}(1 - p_{N,k})$$

$G(N)$ is small **only** if the majority classe has $p_{N,k}$ close to 1

- **Entropy** or cross-entropy

$$H(N) = - \sum_{k=1}^K p_{N,k} \log_2(p_{N,k})$$

↪ $G(N)$ and $H(N)$ are quite similar numerically

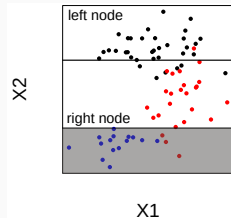
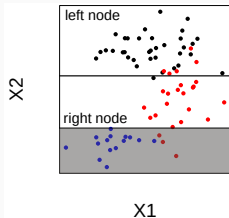
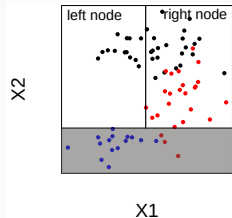
↪ CART with Gini index is the most used technique

Construction: Splitting a node i

Node splitting

- Split a node N into a left and a right child node N_L and N_R
- Child nodes depend on the chosen **feature** j and the **threshold** t for the split

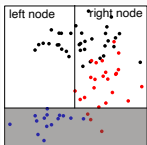
Many possible splits



Construction: Splitting a node ii

Evaluate the quality of a split

- For every possible split of node N into children N_L and N_R , evaluate the **impurity** of the resulting child nodes



$$I(N_L) = 0$$
$$I(N_R) = 0.5$$

where I denotes

- the variance V in the regression context
 - the Gini index G or the entropy H for classification
- compute a **weighted sum of impurity** of both child nodes

$$I(N_L, N_R) = w_L I(N_L) + w_R I(N_R) = \frac{1}{13} 0 + \frac{1}{44} 0.5 = 0.011$$

where the weights are the proportion of data points from node N assigned to the left and right node: $w_L = \frac{\#N_L}{\#N}$ and $w_R = \frac{\#N_R}{\#N}$

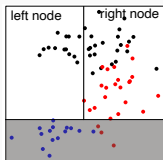
Construction: Splitting a node iii

Best split

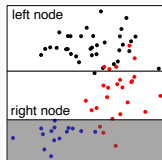
- Among **all** feature-threshold pairs (j, t) (i.e. for any $j \in \{1, \dots, d\}$ and $t \in \mathbb{R}$), the best split **minimizes** the associated **impurity** measure
- Equivalently, the best split **maximizes** the **information gain** defined as

$$IG(j, t) = I(N) - I(N_L(j, t), N_R(j, t))$$

where $I(N)$ is the impurity of node N



$$\begin{aligned} IG &= 0.494 - 0.011 \\ &= 0.483 \end{aligned}$$



$$\begin{aligned} IG &= 0.494 - 0.006 \\ &= 0.488 \end{aligned}$$

Recursive partitioning

- CART method builds the partition iteratively
- starting from the root
- until some stopping criterion is satisfied

Algorithm 1 CART algorithm

```
1: while stopping criterion is not met do  
2:   for every node  $N$  of the current tree do  
3:     Find the best feature-threshold pair  $(j, t)$  that maximizes  $IG(j, t)$   
4:     Create the new child nodes  
5:   end for  
6: end while
```

Greedy approach

- Greedy algorithms make the optimal choice at each step
- No regret strategy on the choice of the splits
- No guarantee to find the optimum

Recall: Training and test sets

- The tree is learned on a **training set**
- Split data $\mathcal{D} = \{(y_i, x_i), i = 1, \dots, n\}$ into a training set and a test set: $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}} = \mathcal{D}$
 - Use $\mathcal{D}_{\text{train}}$ to learn the tree
 - Use $\mathcal{D}_{\text{test}}$ to evaluate error metrics and performance measures

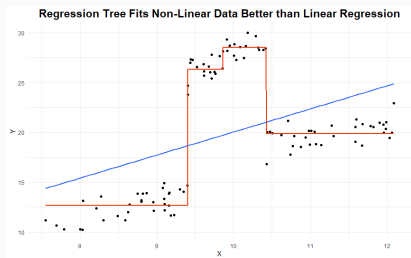
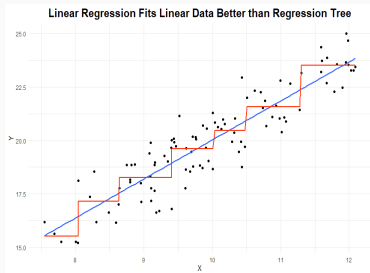
Stopping criteria

- Common criteria
 - Maximum depth of the tree is attained
 - Maximum number of leaves is attained
 - Number of data points per leaf is below a given threshold
 - Impurity per leaf is small enough
 - Test error increases
- Early stopping is always critical

Alternative to stopping criterion

- Grow a maximal tree, then do **pruning**
- Not optimal either

Regression trees versus linear regression



Classification tree versus logistic regression

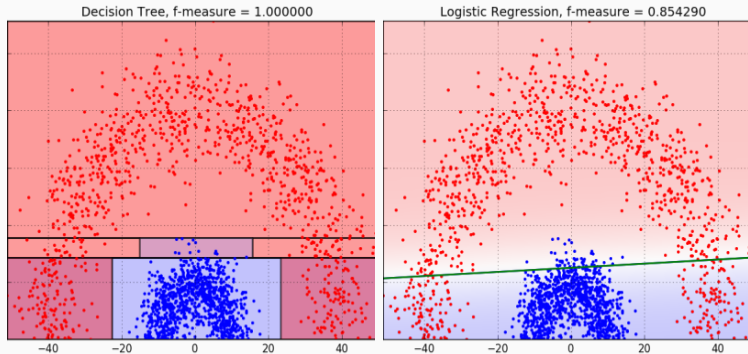
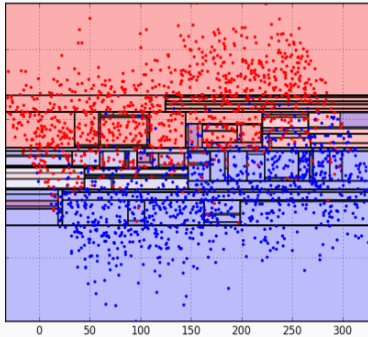
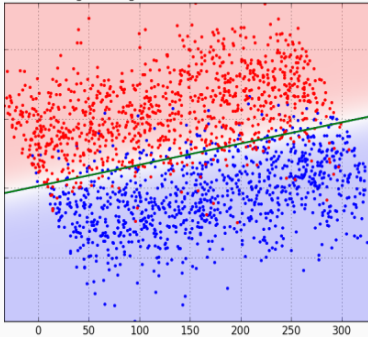


Illustration iii

Decision Tree, f-measure = 0.889780



Logistic Regression, f-measure = 0.922420



Performance of decision trees

- accurate and highly interpretable **on small datasets**
- unstable results **on moderate-size and large datasets**
- **not competitive** with the best supervised learning approaches
- **improvement**: combine multiple trees to a single consensus prediction, called **ensemble methods** as
 - bagging
 - random forests
 - boosting

Bagging

Random forests

Boosting

Explainable AI

- is a set of processes and methods that allows users to understand and trust the results and output created by machine learning algorithms
- provides the explanations accompanying model output to address different concerns and issues as interpretability

Variable importance

- Quantification of importance of a **specific** feature
- Different variable importance measures exist
- Naïvely, a feature j that appears in many splits is important

Feature importance based on information gain

- An overall summary of the importance of feature j
- measured by the **sum of the information gains** obtained over all splits based **on feature j**
- indicates how much including that variable improves the purity of the nodes.
- Beware that this measure does not account for correlations among features
- Python: use `feature_importances_`
- the measure is based on the training set

Mean Decrease Accuracy (MDA)

- measures the decrease in accuracy of the model if the values of one feature are **randomly shuffled**
- randomly permuting the values has the effect of removing all predictive power for that feature
- Python: use `permutation_importance` from `sklearn.inspection` import
- the accuracy is computed from the test data

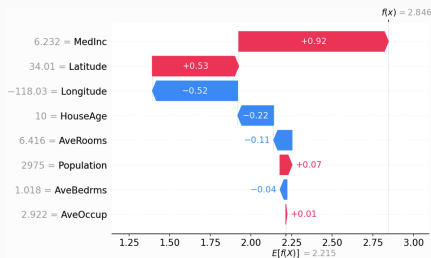
SHAP values

- SHAP = SHapley Additive exPlanations (Lundberg and Lee (2017))
- general concept to quantify **how exactly each feature contributes** to every single prediction made by a model

SHAP values ii

Properties of SHAP values

- Global interpretability: how each feature contributes to the overall model
- Local interpretability: how a particular instance contributed to a specific prediction
- Normalization: the prediction is decomposed as the sum of a baseline value and the SHAP contributions of all features

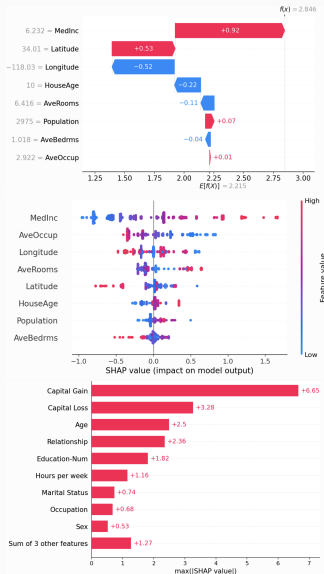


$$\hat{y}(x) = \phi_0 + \sum_{j=1}^d \phi_j$$

- ϕ_j = SHAP value de la variable j
- ϕ_0 = baseline / expected prediction

Visualizing SHAP Values

- bar plot or a feature importance plot, which shows the average absolute contribution of each feature across the dataset
- baseline plots to understand how the features perform compared to the model's baseline
- SHAP value plots allow us to identify important features and gain insights into the decision-making process of the model



Game theory approach

- Features $x_1, \dots, x_d$ are *players* that may (or not) participate in the *collaborative game* of predicting y using model $\hat{y}$
- Denote $v(S)$ the output of the game when only the features given by $S \subset \{1, \dots, d\}$ participate in the game
- The Shapley value v_j for feature j is obtained by
 - comparing outputs v when j participates and when it does not:

$$v(S \cup \{j\}) - v(S)$$

- average over many possible subsets S
- A common choice of the value $v(S)$ is the average output of the model when the values of the features $x_S = \{x_j, j \in S\}$ are known

$$\mathbb{E}[\hat{y}(x)|x_S]$$

SHAP values based on averaging

- Define $v(S)$ as

$$v(S) = \frac{1}{|\mathcal{D}|} \sum_{x_b \in \mathcal{D}} \hat{y}(x_S, x_{S^c}^b)$$

where

- $x_S = \{x_j, j \in S\}$ the values of the features given by S
- x_b is a baseline/background feature vector (and $x_{S^c}^b = \{x_j^b, j \in S^c\}$)

Background values

- use the all zero-baseline ($x_b = 0$) (when features are standardized)
- use a set of background values given by a subset of the data and average over all background values
- Python: `shap.TreeExplainer` with option `feature_perturbation = 'interventional'`

Tree-SHAP

- Tree-based approach
- leverages the tree structure to compute exact SHAP values for tree-based models
- tree-SHAP is both **fast and exact**
- different assumptions can be used to model feature dependence.
- interpretation becomes more subtle when features are highly correlated.
- Python: `shap.TreeExplainer` with option `feature_perturbation = 'tree_path_dependent'`

Bagging

Random forests

Boosting

Inconvenients of decision trees

- CART suffers from
 - low prediction accuracy
 - lack of robustness, high variance: new training datasets give rise to rather different trees
- Decision trees are **weak predictors**

↪ Instead of looking for one optimal tree, grow many, many trees and combine them to a more accurate predictor

↪ Substantial improvements of prediction accuracy can be obtained by aggregating many decision trees (**ensemble methods**)

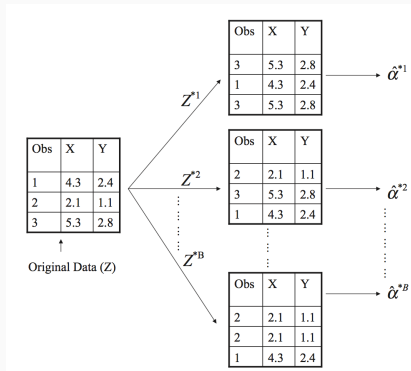
Bagging approach

- General-purpose procedure to reduce the variance of **any statistical learning method**
- Bagging = **B**ootstrap **A**ggregation
- Particularly useful for decision trees
- Principle of **bagging**:
 - create many artificial datasets via **bootstrap**
 - learn many weak predictors on the bootstrapped samples
 - average/combine all predictors to obtain a highly accurate predictor
- **Variance reduction**: Let $Z_1, \dots, Z_n$ be iid with variance σ^2 . The variance of the mean $\bar{Z}_n$ is σ^2/n . In other words, averaging a set of observations reduces variance.

Bootstrap i

Bootstrap

- general **resampling** method to create new artificial datasets
- use bootstrapped samples to **evaluate the accuracy** (e.g. bias, variance, confidence interval) of any estimator or predictor



Bootstrap datasets

- obtained by sampling **with replacement** from the original dataset: randomly choose observations (x_i, y_i)
- generally contain some observations repeatedly
- same size as the original dataset

General bootstrap approach

- Notations
 - $\hat{f}$: any statistical procedure or estimator that can be learned on a sample ; $\hat{f}$ can be thought of an approximation of some “true” value or the “best” method f
 - $\hat{f}_{\mathcal{D}}$: estimator $\hat{f}$ trained on the original data $\mathcal{D}$
 - $\mathcal{D}^{(b)}$, $b = 1, \dots, B$: bootstrap datasets
 - $\hat{f}^{(b)}$: estimator learned on $\mathcal{D}^{(b)}$ (**bootstrap replicates**)
- **The bootstrap replicates $\{\hat{f}^{(b)}\}_{1 \leq b \leq B}$ are a sample of a distribution that is close to the distribution of $\hat{f}$**
- Denote $\bar{f}_B = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}$ the bootstrap mean
- Bootstrap approximation of the **bias** $\mathbb{E}(\hat{f}) - f$ and the **variance** of $\hat{f}$:

$$\widehat{bias}_{\text{boot}}(\hat{f}) = \bar{f}_B - \hat{f}_{\mathcal{D}} \quad \widehat{var}_{\text{boot}}(\hat{f}) = \frac{1}{B} \sum_{b=1}^B \left(\hat{f}^{(b)} - \bar{f}_B \right)^2$$

Algorithm 2 Bagging of regressors or classifiers

- 1: **for** $b = 1, \dots, B$ **do**
 - 2: Sample with replacement $\mathcal{D}^{(b)}$ from the original dataset $\mathcal{D}$
 - 3: Fit a regressor (resp. classifier) $\hat{f}^{(b)}$ on $\mathcal{D}^{(b)}$
 - 4: **end for**
 - 5: Aggregate $\hat{f}^{(1)}, \dots, \hat{f}^{(B)}$ to get a single regressor (resp. classifier) using an average (resp. majority vote)
-

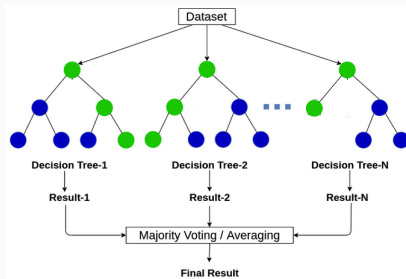
Bagging for decision trees

- Trees are grown deep and not pruned. Each individual has high variance, but low bias.
- Aggregation of trees does not lead to a simple aggregated tree
 $\hookrightarrow$ interpretability gets lost

Bagging of regressors or classifiers ii

Prediction in a bagged model

- For a new point x_{new} , **every** bootstrap tree makes a prediction
- Aggregate all predictions to a single one:
 - in regression: average of all predictions
 - in classification: majority vote



Out-of-bag error

- straightforward way to estimate the **test error** of a bagged model
- no cross validation, no validation set approach
- on average, a bootstrap sample only uses two-thirds of the data
- the remaining one-third are the **out-of-bag (OOB)** observations
- for every (x_i, y_i) , there are around one third of the bootstrap trees that were learned without using this observation

Computation of the out-of-bag error

- For $i = 1, \dots, n$
 - use all bootstrap trees that were learned when (x_i, y_i) was OOB and compute the prediction $\hat{y}_i^{(b)}$ with the corresponding tree
 - average the predicted responses (or take a majority vote) to obtain a single prediction $\hat{y}_i$
- use all predictions $\hat{y}_i, i = 1, \dots, n$ to compute
 - in regression: the mean-squared error
 - in classification: a classification error

↔ The OOB error is a **valid estimate of the test error** for the bagged model

Drawbacks of bagging

- Trees on bootstrap samples are relatively similar $\hookrightarrow$ **correlated trees**
- If, for instance, the data contain a dominant feature for prediction, this feature appears at the top split of all trees
- Bagging has relatively **large variance**: when the whole bagging procedure is repeated on different training sets, quite different predictors are obtained

Bagging

Random forests

Boosting

Random forests

- Extension of bagging
- Aim: reduce variance
- Idea: force trees learned on bootstrap samples to be more different for a better exploration of the model space $\hookrightarrow$ **decorrelate trees**

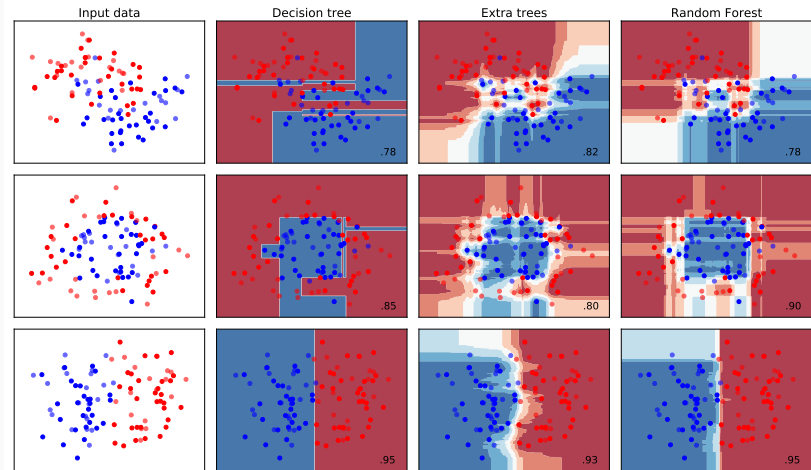
Modification when growing a tree

- Do **column subsampling** also called **feature bagging**
- When looking for splits **only a random subset of features is tried**
- Reduces the correlation between trees, especially when some features are particularly strong

Extremely randomized trees or Extra trees

- A variant of random forests and bagging
- Feature selection for the splits is done in the bagging fashion by minimizing an impurity measure (Gini, Entropy)
- **Thresholds are selected uniformly at random in the feature range**
- Computation is fast
- Leads to accurate predictors

Random forests iii



Bagging

Random forests

Boosting

Ensemble learning

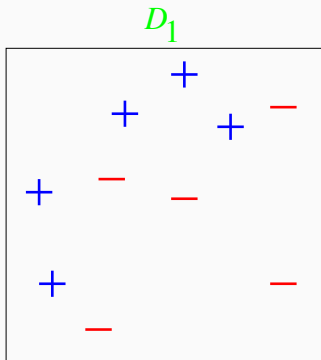
- Idea: combine many weak learners to form a strong one
- Bagging and random forests:
 - use independent learners
 - treat all weak learners equally
- Boosting:
 - weak learners are not independent
 - associate different weights to weak learners

Boosting

- Sequential approach
- Each weak learner depends on the previously built learners
- Idea: correct for mistakes of the current learner
- Reweight the data points before feeding them to the model
- A simple reweighting approach: replicate certain observations in order to focus the classifier on predicting well on them

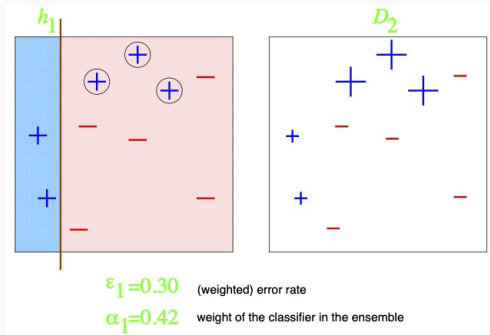
AdaBoost

- = adaptive boosting
- for binary classification of points with labels $+$ ($= 1$) and $-$ ($= -1$)



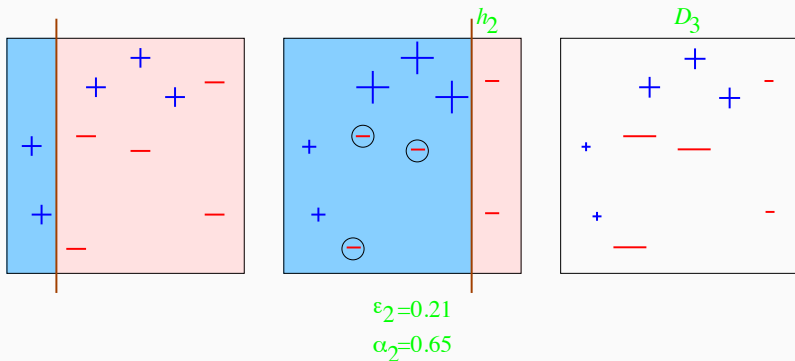
- weak classifiers: vertical or horizontal half-planes
- $D_b = (D_1^{(b)}, \dots, D_n^{(b)})$: weights for data points
- start with equal weights:
 $D_1 = (\frac{1}{n}, \dots, \frac{1}{n})$

First round



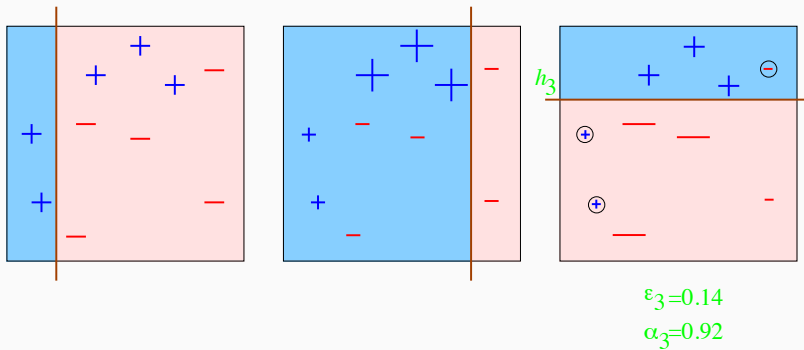
error rate $\epsilon_1 = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{h_1(x_i) \neq y_i\}} = \sum_{i=1}^n D_i^{(1)} \mathbf{1}_{\{h_1(x_i) \neq y_i\}}$ where $y_i \in \{-1, 1\}$ are the true labels

Second round



with $\varepsilon_2 = \sum_{i=1}^n D_i^{(2)} \mathbf{1}_{\{h_2(x_i) \neq y_i\}}$ weighted error rate

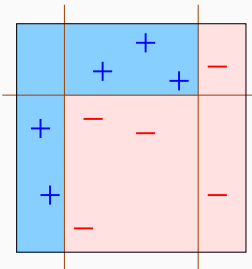
Third round



Final classifier

$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} + 0.65 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} + 0.92 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} \right)$$

=



Denote the **weighted error rate** $\varepsilon(h, D) = \sum_{i=1}^n D_i \mathbf{1}_{\{h(x_i) \neq y_i\}}$

Algorithm 3 AdaBoost

- 1: Initial weights of the data points: $D_1 = (\frac{1}{n}, \dots, \frac{1}{n})$
 - 2: **for** $b = 1, \dots, B$ **do**
 - 3: Train classifier: $h_b \in \arg \min_{h \in \mathcal{H}} \varepsilon(h, D_b)$
 - 4: Error rate: $\varepsilon_b = \varepsilon(h_b, D_b)$
 - 5: Classifier weight: $\alpha_b = \frac{1}{2} \log \left(\frac{1 - \varepsilon_b}{\varepsilon_b} \right)$ (log odds ratio)
 - 6: Normalizing constant: $Z_b = 2 \sqrt{\varepsilon_b (1 - \varepsilon_b)}$
 - 7: New data weights: $D_i^{(b+1)} = D_i^{(b)} \frac{e^{-\alpha_b y_i h_b(x_i)}}{Z_b}$
 - 8: **end for**
 - 9: **return** Boosting classifier: $g_B(x) = \text{sign} \left(\sum_{b=1}^B \alpha_b h_b(x) \right)$
-

If correct prediction $h_b(x_i) = y_i$, that is $h_b(x_i)y_i = 1$, then $e^{-\alpha_b y_i h_b(x_i)} < 1$.

Hence $D_i^{(b+1)}$ decreases

Boosting as a minimization algorithm

- Goal: an ensemble learner of the form

$$g_B(x) = \sum_{b=1}^B \alpha_b h_b(x)$$

that minimizes an empirical risk

$$\frac{1}{n} \sum_{i=1}^n \ell(y_i, g_B(x_i))$$

where ℓ is a loss (least squares, logistic...)

- Minimization over a chosen family $\mathcal{H}$ of weak learners and weights $\alpha_b \in \mathbb{R}$

Gradient boosting

- General-purpose method: many choices for the loss ℓ , many choices of weak learners
- AdaBoost for binary classification:
 - exponential loss $\ell(y, z) = e^{-yz}$
 - decision trees of depth 1, called stumps

Problem

- How to find weak learners h_b and coefficients α_b ?
- Even given $\alpha_1, \dots, \alpha_B \in \mathbb{R}$, minimize over $|\mathcal{H}|^B$ to find the $h_1, \dots, h_B$
- Size of $\mathcal{H}$ is typically $O(d)$ (number of features)

Greedy algorithm

- At iteration $b + 1$: update current learner g_b by

$$g_{b+1} = g_b + \alpha_{b+1} h_{b+1}$$

where h_{b+1} and α_{b+1} are solutions of

$$\arg \min_{\alpha \in \mathbb{R}, h \in \mathcal{H}} \sum_{i=1}^n \ell(y_i, g_b(x_i) + \alpha h(x_i))$$

Still a problem

- Exact minimization at each step is too hard

Gradient boosting idea

- Replace exact minimization by a gradient step
- Approximate the gradient step by an element of $\mathcal{H}$

Algorithm 4 Gradient boosting

- 1: Put $g_0 = \arg \min_{m \in \mathbb{R}} \sum_{i=1}^n \ell(y_i, m)$
- 2: **for** $b = 1, \dots, B$ **do**
- 3: Get gradient directions: $\delta_{b,i} = -\left(\nabla_u \ell(y_i, g_b(x_i) + u)\right)\Big|_{u=0}$
- 4: Find learner h_{b+1} that approximates direction δ_b :

$$(h_{b+1}, \nu_b) = \operatorname{argmin}_{h \in \mathcal{H}, \nu \in \mathbb{R}} \sum_{i=1}^n (\nu h(x_i) - \delta_{b,i})^2$$

- 5: Classifier weight: $\alpha_{b+1} = \arg \min_{\alpha \in \mathbb{R}} \sum_{i=1}^n \ell(y_i, g_b(x_i) + \alpha h_{b+1}(x_i))$
 - 6: Update ensemble learner: $g_b = g_{b-1} + \alpha_b h_b$
 - 7: **end for**
 - 8: **return** Boosting learner g_B
-

Hyperparameters

- Family $\mathcal{H}$ of weak learners:
 - If $\mathcal{H}$ contains decision trees, choose their depth (usually small... no more than four or five)
 - If $\mathcal{H}$ contains generalized linear models (logistic regression), select the number of features (usually one or two)
- Number of iterations B :
 - overfitting is possible (contrary to bagging and random forests)
 - use cross validation, stop when test error does no longer improve
 - usually a few hundreds, few thousands
- Check documentation of `GradientBoostingRegressor` of the `sklearn.ensemble` module for more details on the hyperparameters

Implementation of gradient boosting ii

Many implementations available

... with many clever extra tricks

- GradientBoostingRegressor
 - Part of the `sklearn.ensemble` module
- XGBoost
 - <https://xgboost.readthedocs.io/en/latest/>
 - <https://github.com/dmlc/xgboost>
 - Has been state of the art for years
- LightGBM
 - <https://lightgbm.readthedocs.io/en/latest/>
 - <https://github.com/Microsoft/LightGBM>
 - Faster (and better ?) than XGBoost
- CatBoost
 - <https://catboost.ai/en/docs/>
 - One of the most recent implementations
 - Adapted for categorical features

To make models work

For standard tabular data (no image, signal, text etc.)

- Spend time on **feature engineering**
- Spend time on **feature engineering**
- ...
- Spend time on **feature engineering**
- Always try out random forests or gradient boosting **before** diving into a deep learning method (tomorrow)

Thank you!